Constitutional Invariants for Governing Autonomous AI Action

Art Harol
Ambit Systems
art.harol@ambit-systems.com

April 2026

Abstract

Autonomous AI systems now execute actions with real-world consequences, but current governance mechanisms address only fragments of the problem: some intervene after consequence, some govern at the wrong boundary, and some do not produce evidence that survives independent verification. The problem is architectural. This paper defines twelve constitutional invariants for governing autonomous AI action at execution. The invariants are stated within a formal model in which a deterministic evaluation function computes an explicit decision from a canonical action representation, validated delegation artefacts, and an immutable policy identity. Under this model, violating any invariant reopens a specific governance failure mode in the execution path. We evaluate representative systems against the invariant set and find that each satisfies a subset, while none in the reviewed corpus satisfies the full set. The gap is therefore structural: fragment-wise satisfaction of governance properties does not compose into governance. Five open problems separate constitutional specification from verifiable enforcement: conformance specification, authority bootstrap, adapter trust, composition governance, and adversary modelling.

1 Introduction

When an autonomous AI system transfers funds from a corporate account, deletes a production database, or sends a communication to an external party, a question arises that current infrastructures still struggle to answer reliably: *was this specific action authorised, under which delegation, under which policy version, with evidence that survives independent scrutiny?*

Classical computer security assumes three conditions that autonomous AI systems violate. First, that a human makes or approves each consequential decision. Second, that identity-bound permissions adequately scope authority. Third, that the executing entity follows a deterministic code path whose behaviour can be reviewed in advance. When the executing entity reasons probabilistically, generates novel action sequences, and operates across trust boundaries through tool composition, all three assumptions break simultaneously.

The resulting governance gap is concrete: within the representative literature reviewed here, we did not find a mechanism that evaluates every consequence-bearing action before execution, under explicit delegation, through a deterministic function with independently verifiable evidence of its decision. Runtime interception toolkits, delegation protocols, model guardrails, and logging infrastructure each address fragments of this requirement. In the reviewed corpus, none satisfies it in full. The problem cannot be solved by extending or hardening any of these mechanisms in isolation; it requires defining a set of invariants that, on this account, a valid governance system must satisfy, and evaluating existing and future systems against them. This paper argues that

those fragmented requirements can be made explicit as a coherent invariant set, and that, under the model developed here, claims to governance of autonomous AI action require the set rather than any individual invariant.

Under this framework, a system that violates any invariant is, with respect to that action, ungoverned under this model: the violated invariant identifies the specific failure mode.

Contributions. This paper makes three contributions:

1. It proposes twelve constitutional invariants that, on the formal model developed here, any system governing autonomous AI action must satisfy (§3).
2. It shows, within a criterion-based review of representative systems, that existing systems satisfy invariants in isolation but do not compose into a complete governance model (§4).
3. It identifies five open problems — conformance specification, authority bootstrap, adapter trust, composition governance, and adversary modelling — that separate constitutional specification from verifiable enforcement (§5).

Novelty. Invariants 1, 3, and 7 adapt established security principles — complete mediation [33], capability-carried authority and least-authority discipline in capability systems [12, 26], and fail-safe defaults [33] — to the autonomous AI action boundary. The following elements are, to our knowledge, original contributions of this paper. First, the resolution/evaluation separation (Definitions 4–5) builds on the attribute-retrieval/decision separation present architecturally in XACML's PIP/PDP model [28] and sharpens it into a mandatory purity boundary: all I/O completes before evaluation begins, and resolved facts carry integrity evidence. Second, canonical replay determinism (Property 2) requires byte-identical evidence payloads from independent evaluators. Third, composition closure is stated as a constitutional invariant (Invariant 11), elevating an empirical observation [32] into a governance condition argued as necessary within this model. Finally, Section 4 advances the fragmentation finding that satisfying invariants independently does not compose into governance.

Scope. The invariants govern *actions at the action boundary* — the point where intent becomes external consequence. They do not govern model internals, inference-time reasoning, information flows not mediated by tool invocation, or workflow-level transactional atomicity. These are distinct problems requiring distinct mechanisms. The claim is governance of autonomous AI action, not complete governance of autonomous AI systems.

Adversary model. The invariants must hold under (1) malicious or misaligned agent behaviour, (2) compromised or misbehaving components in the execution path, (3) partial system failure or missing context at evaluation time, and (4) authorised operators who weaken governance under production pressure. Section 3.3 defines the adversary classes.

Non-goals. This paper does not guarantee correctness of agent reasoning, absence of harmful intent, or workflow-level atomicity. It governs whether specific actions are authorised before execution and whether that authorisation is independently verifiable.

The remainder of this paper is organised as follows. Section 2 traces the historical lineage from capability-based security to the emerging AI governance landscape. Section 3 presents the formal model, adversary model, and twelve invariants. Section 4 evaluates the field against the invariants. Section 5 identifies five open research problems. Section 6 discusses limitations. Section 7 concludes.

2 Background and Related Work

2.1 Historical Lineage

The architectural commitment underlying this paper — that authority must be carried with the actor, scoped to a specific context, and verified at the point of action — has a sixty-year lineage in systems security.

Dennis and Van Horn [12] introduced a capability-based protection model in which possession of a capability confers the ability to access an object according to the rights encoded in that capability. Saltzer and Schroeder [33] formalised the principle of *complete mediation* — every access to every object must be checked for authority, not merely at session creation — and the principle of *least privilege*, which capability systems enforce structurally rather than administratively.

Hardy [17] gave the tradition its most durable failure mode: the *confused deputy*, a program that cannot distinguish its own authority from the authority of its caller and inadvertently exercises the wrong one, creating unintended authority paths. The confused deputy is not a bug in any individual access grant; it is a structural consequence of ambient authority. In autonomous AI systems, tool composition creates an analogous failure at scale: an agent acting on one tool's output may inadvertently exercise another tool's authority, and the resulting privilege escalation exploits topology, not any individual vulnerability.

Miller [26] systematised these insights in the *object-capability* tradition: authority flows through references held by objects, delegation occurs by passing references, and least authority concerns the authority available to a computation at any moment. That distinction maps directly to the governance of autonomous AI action.

seL4 [21] carried the verification ambition to its conclusion: a machine-checked proof that the C implementation of a capability-based microkernel correctly refines its abstract specification. Subsequent work extended this to integrity and information-flow enforcement properties, and the DARPA HACMS programme [15] demonstrated resilience of the verified kernel under adversarial red-team testing in a realistic operational context. Its limitation for autonomous AI governance is that its authority model is fixed at design time: capability derivation is configured by a trusted initialiser, not established dynamically between autonomous actors with temporal validity, revocation, and scope constraints.

The common thread is a single architectural commitment: authority must be carried, not inferred; scoped, not ambient; and verified at the boundary where intent becomes consequence. The invariants presented in Section 3 extend this commitment to autonomous AI systems, where delegation is dynamic, revocation must be fresh, composition creates emergent authority, and every decision must produce independently verifiable evidence.

The capability tradition is one branch of a broader lineage. Anderson's reference monitor concept [2] — a mediation mechanism intended to be tamper-resistant, always invoked, and amenable to verification — directly anticipates the enforcement boundary defined in this paper (Definition 4). Schneider [34] characterised which security policies are enforceable by execution monitoring — exactly the safety properties — providing the theoretical basis for runtime enforcement at the action boundary. Clark and Wilson [7] formalised integrity invariants for commercial systems: well-formed transactions that preserve data integrity through constrained interfaces, a structural parallel to governance evaluation through canonical action representations. The canonical action (Definition 1) shares structure with attribute-based access control request contexts, notably XACML [28], but extends them with delegation artefacts and policy identity as first-class evaluation inputs. The conformance problem (Problem 5.1) parallels the

assurance levels of the Common Criteria evaluation methodology [9], which similarly asks: given a specification, what does it mean for an implementation to satisfy it?

2.2 Emerging Landscape

The period from late 2024 to early 2026 has seen rapid growth in academic and industry work on governing autonomous AI systems. We organise this landscape by the governance dimension each work addresses, rather than by publication, to expose convergence and gaps.

Runtime enforcement. Several systems interpose a policy enforcement layer on the execution path. Microsoft's Agent Governance Toolkit [25] intercepts agent actions before execution via a policy engine, achieving sub-millisecond evaluation latency. It hooks into multiple agent frameworks (LangChain, CrewAI, Google ADK) and claims coverage of the OWASP Agentic Top 10 [29], but bundles identity, governance, sandboxing, and observability into a single toolkit — collapsing governance into the adjacent systems that Invariant 8 requires it to remain distinct from. Aegis [24] offers a close architectural analogue to execution-boundary enforcement: an immutable Ethics Policy Layer bound at genesis, a verification agent, and an enforcement kernel module that triggers autonomous shutdown on violations. It shares the constitutional-supremacy instinct (immutable policy, quorum amendment) but couples governance, identity, and logging into an integrated kernel and frames enforcement as "ethics" rather than authorisation. Progent [36] provides a domain-specific language for defining granular tool-call access policies enforced at runtime, reporting substantial reductions in attack success rate on AgentDojo under both manual and LLM-generated policy settings. SAGA [38] enforces access-control policies on inter-agent communication through a centralised provider, with cryptographic token derivation for fine-grained access. All four systems address pre-execution interception (Invariant 1) but, among these four, none fully satisfies evidence integrity (Invariant 6), none fully addresses delegation as a first-class governance concept (Invariant 4), and none addresses composition-emergent authority (Invariant 11).

Delegation protocols. South et al. [37] describe Agent-ID and Delegation Tokens extending OAuth 2.0/OIDC and present a path from natural-language permissions to auditable access-control configurations. Tomasev et al. [39] develop an adaptive delegation framework covering authority transfer, accountability, role specification, and mechanisms for establishing trust. The paper identifies privilege attenuation as a requirement for recursive task delegation and sketches what it calls Delegation Capability Tokens, based on Macaroons [5] and Biscuits [11], as a future direction, but does not formally specify or implement them. Prakash [30] specifies Invocation-Bound Capability Tokens using JWT [18] (single-hop) and Biscuit/Datalog [11] (multi-hop delegation), with reference implementations that reject all six pre-defined attack categories in controlled testing — including two (depth violation, audit evasion) that standard JWT cannot detect. (Prakash addresses the delegation mechanism rather than governance architecture; it is discussed here for context but is not included in the scored evaluation corpus of Section 4.) These works validate that delegation-as-protocol is entering both the academic and standards vocabularies. However, all three primarily address the delegation *mechanism* rather than the full governance-evaluation architecture used in this paper: the token or chain exists, but the published designs stop short of specifying canonical action evaluation against delegation artefacts with independently reconstructible evidence.

Evidence and integrity. Kao [19] proposes constant-size cryptographic evidence structures with game-based security proofs, making it a close analogue to the evidence record concept formalised in Invariant 6. Rajagopalan and Rao [31] propose authenticated workflows in which operations carry valid cryptographic proof or are rejected, paralleling fail-closed enforcement (Invariant 7). Balan et al. [4] propose a framework for cryptographic verifiability of end-to-end

AI pipelines and argue that cross-stage linkability is essential. These works advance evidence and proof mechanisms, but they do not fully specify the action-boundary governance contract used here: what canonical inputs were evaluated, under which policy identity, and how an independent party reconstructs the decision from the evidence alone.

Composition risk. Reid et al. [32] at the Gradient Institute provide an independent analogue to Invariant 11: their headline finding — *"a collection of safe agents does not guarantee a safe collection of agents"* — is Invariant 11 stated as analytical observation rather than constitutional axiom. They catalogue six composition failure modes (cascading reliability failures, inter-agent communication failures, monoculture collapse, conformity bias, deficient theory of mind, mixed motive dynamics) and provide risk assessment techniques, but stop at risk *identification*. The paper has no enforcement architecture, no evidence integrity model, and no delegation mechanism. It diagnoses composition risk; it does not govern it.

Trust and authorisation mismatch. Shi et al. [35] survey the recent literature on trust-authorisation mismatch and identify the core structural problem: static permissions are decoupled from runtime trustworthiness. They introduce the Belief-Intention-Permission framework and advocate dynamic, risk-adaptive authorisation. Xu et al. [41] introduce an "authorisation drift" metric — defined as the weighted variance of over-exposure rate across trust levels — that quantifies sensitivity to trust-level changes, a correlate of the constitutional prohibition on drift (Invariant 10).

Deterministic policy. Kaptein et al. [20] formalise compliance policies as deterministic functions over agent identity, partial execution path, proposed next action, and organisational state, outputting a policy-violation probability that is subsequently calibrated into discrete interventions. The probabilistic intermediate distinguishes their model from the strict ALLOW/DENY/ESCALATE trichotomy of Invariant 3; evidence integrity is not addressed.

Standards and regulation. The institutional landscape is in motion. NIST [27] has launched an AI Agent Standards Initiative, signalling institutional recognition that autonomous agent systems require dedicated interoperability, identity, and security work. The EU AI Act [14] establishes obligations for high-risk AI systems but provides no agent-specific implementing guidance. The OWASP Agentic Top 10 [29] organises risks rather than specifying a governance architecture. We are not aware of a ratified ISO/IEC standard that defines agent-specific governance at the action boundary in the sense formalised here. The regulatory and standards landscape remains active but unsettled.

Landscape pattern. The emerging literature converges on fragments of the governance problem: runtime interception, delegation protocols, evidence structures, composition risk, deterministic policy. Within the corpus reviewed here, no work satisfies the complete invariant set. Section 4 evaluates this fragmentation through a criterion-based review of representative works.

3 Constitutional Invariants

This section presents the formal model and twelve invariants. The invariants are constitutional in the following sense: they define what governance of autonomous AI action must not violate.[1] They are constraints, not preferences. Within this model, when a mechanism is classified as not satisfying governance, it is because it violates one of these invariants, not because it is unfashionable or incomplete.

[1] We use "constitutional" in the systems sense — invariants that implementations must not violate — distinct from the "Constitutional AI" training methodology [3], which constrains model outputs through reinforcement learning from AI feedback (RLAIF). The invariants here govern runtime action evaluation, not model behaviour during training.

The invariants do not prescribe an implementation. They define what implementations must not violate.

3.1 What Makes an Invariant Constitutional

An invariant is constitutional in this paper if it has five properties. First, it is a necessary condition for the possibility of governance, not an implementation choice: if it is absent, a governance claim fails in principle rather than merely degrading in quality. Second, it is violation-complete: its absence reopens a concrete governance failure mode identifiable in the execution path (Table 1). Third, it is architecture-defining: it constrains the structure by which authority, evaluation, enforcement, and evidence are related, not merely the behaviour of a particular subsystem. Fourth, it is implementation-independent: multiple architectures may satisfy it, and the paper does not prescribe a single mechanism for doing so. Fifth, it is non-composable in fragments: partial satisfaction of several invariants does not amount to satisfying the invariant set, because each missing invariant reopens a distinct failure mode.

On this account, the paper is not proposing features. It defines the conditions under which a claim to govern autonomous AI action is valid.

3.2 Formal Model

Governance of autonomous AI action is the discipline that evaluates intent at the action boundary and renders an explicit, authoritative decision before execution. We formalise this as follows.

The model decomposes governance evaluation into three distinct inputs: the canonical action (the attempted operation together with the resolved contextual facts required to evaluate it), the delegation artefact set (what authority is claimed), and the policy identity (what decision logic applies). This factoring is deliberate. The action representation is independent of who has authority to perform it; delegation is a claim validated against the action, not a property of it; and policy must be versioned and identified independently of both, because reproducibility requires knowing exactly which decision logic was applied. The last definition — resolution — separates the preparation of these inputs, which may require I/O and state-dependent resolution, from the pure evaluation that consumes them. This separation is what makes deterministic governance compatible with a stateful world. The formal model proceeds in three stages: the canonical inputs to governance evaluation (Definitions 1–3), the evaluation function and its enforcement (Definitions 4–5), and the conformance properties that implementations must satisfy (Properties 1–2). Figure 1 shows the runtime sequence.

Definition 1 (Canonical Action)**.** A *canonical action* I is a tuple representing an attempted operation at the point of governance evaluation:

$$I = (\alpha,\ \iota,\ \tau,\ \pi,\ \sigma,\ t)$$

where α is the actor identity, ι is the intent (the operation requested), τ is the target resource, π is the operation's declared parameter set, σ is the resolved context (sequence-derived facts, environmental state, and any other context produced by the resolution phase — see Definition 5), and t is the evaluation timestamp. Two canonical actions are identical when they agree on every component.

Scope. Governance evaluates actions capable of producing *external consequences*: state changes, communications, or resource mutations that cross the boundary of the requesting agent's own execution environment and affect shared or external systems. Examples include API calls, database writes, file-system mutations, network transmissions, and inter-process signals that alter

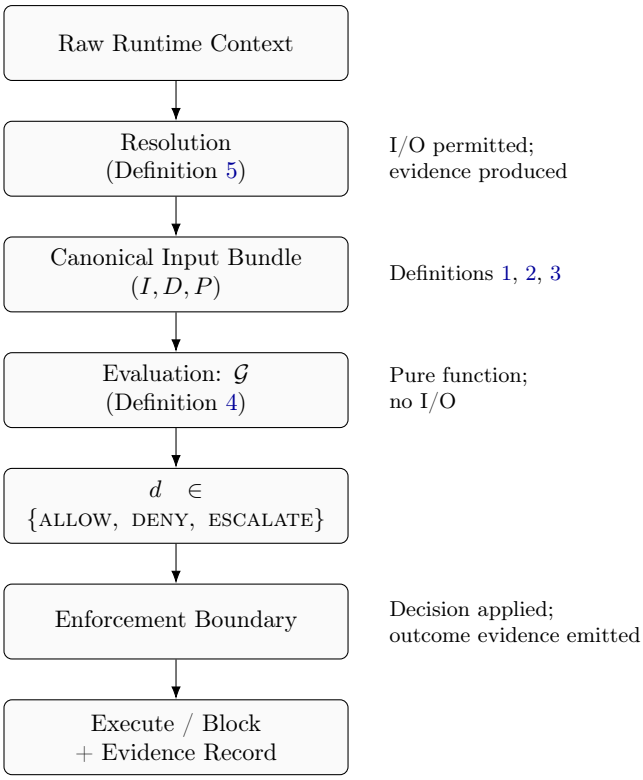

Figure 1: Runtime sequence of governance evaluation. Resolution transforms raw context into the canonical input bundle through I/O-bearing preparation. Evaluation consumes the bundle as a pure function and produces exactly one decision. The enforcement boundary applies the decision and emits outcome evidence cryptographically linked to the decision record.

state visible to other principals. Actions whose effects are confined entirely within the agent's local, ephemeral computation (e.g., intermediate reasoning steps, local variable assignments) are outside scope. The precise demarcation of the action boundary is a deployment-time binding decision, analogous to how capability systems leave the definition of "resource" to the deployment; the invariants require that the boundary be explicitly declared, that the declaration be versioned and auditable as a governance artefact subject to Invariant 10, and that all paths crossing it pass through governance evaluation. The timestamp t must be asserted by a cryptographically authenticated time authority (e.g., RFC 3161 time-stamp authorities [1] or equivalent); this is a resolution requirement (Definition 5), not a structural property of the tuple.

Definition 2 (Delegation Artefact)**.** A *delegation artefact* δ is a tuple:

$$\delta = (s,\ b,\ r,\ v,\ c)$$

where s is the authority scope, b is the temporal validity bounds, r is the revocation status, v is the version identifier, and c is the delegation-chain provenance (an ordered sequence of delegator links) associated with δ. Two delegation artefacts are identical when they agree on every component.

The *delegation artefact set* D is a set of delegation artefacts presented for evaluation. $D = \emptyset$ represents the absence of delegation.

Admissibility. Artefacts in D must be explicit (not inferred from ambient context) and validated during resolution (Definition 5). Revocation status must be resolved before evaluation and be cryptographically verifiable. Invariant 4 specifies the full constitutional requirements on delegation.

Definition 3 (Policy Identity)**.** A *policy identity* P is an identifier that uniquely and permanently determines the decision logic applied during evaluation. Given P, the decision logic applied by the governance evaluation function (Definition 4) is fully determined. References to "latest" or "current" do not constitute policy identity.

Versioning. Distinct policy versions must receive distinct identifiers. An evidence record issued under P_k must remain replayable under the policy definitions referenced by P_k regardless of subsequent policy evolution (Invariant 10).

With the three inputs to governance evaluation defined, we now specify the function that consumes them and the enforcement boundary that applies its decisions.

Definition 4 (Governance Evaluation)**.** The *governance evaluation function* $\mathcal{G}$ is a total, pure function that maps every canonical action, delegation artefact set, and policy identity to exactly one decision:

$$d = \mathcal{G}(I, D, P) \quad \text{where} \quad d \in \{\text{ALLOW}, \text{DENY}, \text{ESCALATE}\}$$

$\mathcal{G}$ performs no I/O, consults no external state, and introduces no non-determinism. Operational failure conditions are represented within the codomain by DENY; totality means that evaluation never yields an undefined result.

Decision semantics. ALLOW authorises execution; DENY prohibits it; ESCALATE transfers decision authority to a defined higher-authority process without executing the action (Invariant 3).

Enforcement. The *enforcement boundary* is the mechanism that intercepts every execution path capable of producing external consequences, submits it to resolution and evaluation, and blocks or permits execution in accordance with the decision. Its behaviour is determined by d: ALLOW permits execution, DENY blocks it, and ESCALATE defers to a higher-authority process. The enforcement boundary must not alter, reinterpret, or override the decision. For every external consequence, the enforcement boundary emits an outcome record cryptographically linked to the corresponding decision record, enabling independent verification that what executed matches what was authorised (Invariant 6).

The canonical input bundle (I, D, P) presented to $\mathcal{G}$ is not the raw runtime request. A *resolution phase* transforms raw runtime context into the canonical inputs that evaluation consumes.

Definition 5 (Resolution)**.** *Resolution* transforms raw runtime context into the canonical input triple (I, D, P). Resolution may perform I/O — canonicalisation, delegation validation, revocation resolution, sequence context derivation, timestamp acquisition — but must produce attributed, versioned, reviewable facts included in the evidence record (Invariant 6). Resolution must not make governance decisions, silently omit governance-relevant context, or introduce unexplained variation into the canonical input. Canonical abstraction — reducing runtime detail to a governance-relevant representation — is necessary and expected; silent omission of facts that governance rules could act on is a resolution failure. Failure to establish that the canonical input satisfies the required representation properties is likewise a resolution failure and must resolve to denial under Invariant 7. Any variation in the canonical input must be attributable to explicit, versioned, replayable facts produced during resolution and recorded in the evidence record. Unresolvable context must cause the system to deny the action before execution; no unresolved input may be passed to $\mathcal{G}$.

The component that performs resolution for a specific runtime environment — translating its native tool calls, API requests, or framework events into canonical form — is an *adapter*. An adapter is a member of the trusted computing base: if it produces an unsound canonical representation, governance evaluates a fiction (Problem 5.3).

The separation of resolution (which may perform I/O) from evaluation (which is pure) is the central architectural move. It makes deterministic evaluation achievable without pretending that

the real world is side-effect-free. This separation also exposes the primary epistemic limitation of the entire approach: governance is only as sound as its canonical representation of reality, and the invariants govern the evaluation phase — where determinism is achievable — while the harder problem of ensuring resolution fidelity remains an open research problem (Problem 5.3).

The following two properties define conformance for implementations of $\mathcal{G}$.

Property 1 (Replay Equivalence)**.** For any two independent evaluators $\mathcal{G}_1, \mathcal{G}_2$ conforming to the decision logic identified by the same policy identity P, identical inputs must produce identical decisions:

$$\mathcal{G}_1(I, D, P) = \mathcal{G}_2(I, D, P) \quad \text{for all identical canonical input bundles } (I, D, P)$$

Conformance. Replay equivalence is a conformance requirement derived from Definitions 4 and 3: if P uniquely determines the decision logic and $\mathcal{G}$ is pure, any two correct implementations must agree on all inputs under the same P.

Property 2 (Canonical Replay Determinism)**.** This property extends replay equivalence (Property 1) from decision agreement to evidence reproducibility. The *canonical replay payload* of an evidence record — comprising the canonical action (including resolved context), delegation artefacts, policy identity, canonicalisation specification identifier, and decision outcome — must be byte-identical across independent conforming evaluators given the same inputs. Deployment-specific attestation (chain position, signing key, record timestamp) may differ; the canonical replay payload must not. In particular, the resolved context σ included in the canonical replay payload must contain only facts derived from the canonicalisation specification — facts that any conforming resolution process would produce from the same raw runtime context. Deployment-specific metadata (node identity, infrastructure routing, local clock skew) is attestation, not resolved context, and is excluded from the canonical replay payload.

Canonicalisation scope. A *canonicalisation specification* is a versioned, immutable document that defines the transformation rules and serialisation format used by resolution to produce the canonical input bundle from raw runtime context. Its identifier, included in the canonical replay payload, governs resolution, not evaluation: it records how the canonical input bundle was derived, enabling independent verification of resolution fidelity. It is not an input to $\mathcal{G}$. Byte-identical serialisation of the full payload — including the delegation artefact set D, which has no inherent ordering — requires that the canonicalisation specification define a canonical serialisation for every component.

Properties 1 and 2 are constitutional requirements, not implementation aspirations. Property 2 strictly strengthens Property 1: byte-identical payloads entail identical decisions, but not conversely. If two evaluators diverge on identical inputs under the same policy, at least one is non-conforming to the specified decision logic; non-determinism is one possible cause, but deterministic implementation error is another. The evidence record structure referenced above is formally specified in Invariant 6.

3.3 Adversary Model

The invariants must hold under adversarial conditions. We define four adversary classes that the invariants jointly defend against, ordered by their relationship to the governance boundary. Unless stated otherwise, the adversary is assumed to control components outside the minimal trusted computing base comprising resolution, evaluation, the enforcement boundary, and the evidence-integrity mechanisms; compromise within that base is treated explicitly below.

A1: Malicious or misaligned agent. An agent that generates actions intended to exceed its delegated authority, exploit composition to create unauthorised authority paths, or craft inputs designed to produce incorrect governance decisions. Capabilities: arbitrary action generation within the agent's execution environment; observation of prior governance decisions; ability to coordinate with other agents through shared state. Defended by: Invariants 1 (pre-execution interception), 2 (faithful representation), 4 (scoped, revocable delegation), 5 (non-determinism cannot be exploited to obtain different outcomes on equivalent inputs), 6 (actions without corresponding evidence records are detectable), 7 (governance failure during evaluation cannot be exploited to permit execution), 11 (composition-emergent authority governed).

A2: Compromised adapter or resolution process. A resolution component that produces unsound canonical representations — omitting governance-relevant parameters, distorting values, or fabricating resolved context. This adversary operates within the trusted computing base. Capabilities: arbitrary modification of the canonical action I or resolved context σ presented to $\mathcal{G}$. Defended by: Invariants 2 (representation properties as verifiable requirements), 5 (replay equivalence enables independent re-evaluation of the canonical input), 6 (resolution facts recorded in evidence), 7 (unresolvable context must deny). This is the weakest defence in the model: if the adapter is compromised, governance evaluates a fiction (Problem 5.3).

A3: Evidence tamperer. An adversary who attempts to forge, modify, or selectively omit evidence records to conceal unauthorised actions or fabricate authorisation for actions that were denied. Capabilities: access to the evidence store; ability to insert, modify, or delete records. Defended by: Invariant 6 (tamper-evident chaining, collision-resistant hashing, EUF-CMA signatures [16], independent verifiability), Invariant 7 (missing evidence treated as governance failure), Invariant 10 (historical evidence remains interpretable under its original policy identity, preventing semantic reinterpretation of tampered records).

A4: Governance erosion operator. An authorised operator who weakens policy under production pressure — relaxing time bounds, expanding delegation scope, disabling fail-closed enforcement — until the governance layer is present but permissive enough to be meaningless. This adversary class does not appear in traditional access-control threat models. Capabilities: legitimate policy modification authority. Defended by: Invariant 10 (all changes explicit, versioned, reviewable), Invariant 12 (invariants override operational expediency), Invariant 6 (historical records remain interpretable, making the erosion trajectory auditable).

Policy-semantic exploitation — an adversary who crafts actions that comply with policy constraints while violating their intent — is a real threat but operates against policy content, not the governance mechanism. The invariants ensure the policy is applied consistently; they do not ensure the policy is adequate (see Discussion, *Policy correctness*).

This adversary model is preliminary. Problem 5.5 identifies the open question of whether a complete threat model can be derived from the invariants and whether the residual attack surface under each adversary class is precisely characterisable. The Dolev-Yao model [13] and capability confinement [26] provide relevant formal foundations; extending them to the governance setting is future work.

3.4 The Twelve Invariants

Governance of autonomous AI action fails in two fundamental ways: an unauthorised action executes, or an authorised action cannot be proven as such. The twelve invariants below are presented as a jointly necessary set of constraints for closing both failure modes. The argument is structural, not formal: for each invariant, we identify the specific governance failure that

its absence reopens (Table 1). We do not prove minimality or completeness (see Discussion, *Minimality and completeness*).

Derivation by successive failure closure. The derivation traces a forcing chain: each invariant is motivated by the specific governance failure that remains open even when all preceding invariants are satisfied.

The first failure mode — unauthorised execution — forces a sequence of increasingly specific requirements. Without any governance, actions execute unchecked; pre-execution evaluation is required because post-hoc analysis cannot prevent the consequence it analyses (forcing Invariant 1). But evaluation must operate on something faithful: if the canonical representation omits governance-relevant facts or distorts values, evaluation governs a fiction, and determinism amplifies the error (forcing Invariant 2). A faithful representation must yield an unambiguous outcome: if the decision space admits silence or undefined cases, implicit approval is structurally fail-open (forcing Invariant 3). An explicit outcome must be rendered against explicit authority: if delegation is ambient — inferred from identity or runtime context — the system cannot distinguish authorised from unauthorised exercise of a capability, reproducing the confused deputy [17] at tool-composition scale (forcing Invariant 4). And the decision must be reproducible: if hidden mutable state causes two evaluators to disagree on identical inputs, independent verification is impossible and audit is fiction (forcing Invariant 5).

The second failure mode — an authorised action that cannot be proven as such — forces the evidence requirement: without a tamper-evident, independently verifiable record linking the canonical inputs to the decision, governance is assertion indistinguishable from its absence (forcing Invariant 6).

Invariants 1–6 govern the evaluation path when it succeeds. The remaining invariants close system-level failure modes that this path leaves open. Governance that silently degrades under failure — inability to form canonical inputs, evaluate policy, or verify delegation — is ungoverned precisely when governance is most needed (forcing Invariant 7). Governance whose semantics change when an adjacent system changes — a model upgrade, a logging reconfiguration, a framework migration — has drifted without a governance change being made (forcing Invariant 8). Provider substitution is a specific and common case of this coupling; because autonomous AI systems are deployed across heterogeneous vendor stacks, it merits explicit treatment (forcing Invariant 9). Even with separation, governance can drift through undocumented internal changes — relaxed time bounds, expanded delegation scopes, softened policy rules — and unless all changes are explicit, versioned, and reviewable, historical evidence becomes uninterpretable under its original policy (forcing Invariant 10). Per-component governance does not govern the composition: when individually governed systems interact, the interaction creates authority paths that no individual evaluation assessed (forcing Invariant 11). And without a constraint that these invariants override convenience, performance, and operational expediency, each is individually negotiable and collectively erodible through accommodation (forcing Invariant 12).

The invariants are organised into four clusters: *boundary and decision* (Invariants 1–3), *authority* (Invariant 4), *evaluation and evidence* (Invariants 5–6), and *system properties* (Invariants 7–12). Each invariant is stated, formalised, and accompanied by the governance failure that occurs in its absence.

3.4.1 Boundary and Decision

Invariant 1 (Governance Scope)**.** Governance evaluates autonomous action before consequence. It does not generate intent, plan workflows, execute actions, or route traffic. It does not replace identity, logging, or network controls.

Formal property. For every execution path capable of producing external consequences, an evaluation $\mathcal{G}(I, D, P)$ must temporally precede execution. No consequence-producing path may execute without a prior governance decision.

Failure mode. Without pre-execution evaluation, governance is post-hoc analysis — useful for incident response but unable to prevent the consequence it analyses. Post-execution controls do not satisfy this invariant. Detecting that an action was wrong after it executes may be valuable, but it does not constitute governance as defined here.

Dependencies. Foundation invariant. Required directly by Invariants 2–4, and indirectly by all invariants that depend on them.

Invariant 2 (Action Boundary Primacy)**.** Governance evaluation operates exclusively on the canonical input bundle (I, D, P). Within that bundle, the canonical action component I must satisfy five representation properties stated below. All execution paths capable of producing external consequences must pass through governance evaluation.

Formal property. The canonical action component I must satisfy five properties:

(i) *Completeness*: the representation preserves all governance-relevant facts from the runtime action. A missing field that governance rules could act on is a representation failure.
(ii) *Soundness*: the representation does not invent or distort meaning. When a value cannot be determined, an explicit unknown indicator is required, not a plausible default.
(iii) *Stability*: equivalent runtime behaviour maps to equivalent canonical forms.
(iv) *Provenance*: every field in I is traceable to its runtime source through declared transformation rules.
(v) *Binding*: there exists a mechanical linkage between I and the captured request intended for execution. Binding to the *observed execution outcome* is governed by Invariant 6 (consequence markers) and is not guaranteed by representation alone.

Anti-bypass. Every execution path capable of producing external consequences must pass through $\mathcal{G}$. Any architecture that permits bypass of governance evaluation violates this invariant.

Failure mode. If the representation is deficient, deterministic evaluation governs a fiction, evidence integrity records that fiction faithfully, and anti-bypass cannot detect it. A system that produces deterministic decisions from unsound representations is not governed — it is wrong in a reproducible way.

Dependencies. Requires Invariant 1. Required by Invariants 5, 6, 11.

Invariant 3 (Explicit Decision)**.** Every evaluated action results in exactly one explicit outcome: ALLOW, DENY, or ESCALATE. Silence is not permission. If evaluation cannot complete, the outcome is DENY.

Formal property. $\mathcal{G}$ is total (Definition 4): every input produces exactly one of ALLOW, DENY, or ESCALATE. There is no fourth outcome. ESCALATE does not execute the action; it transfers decision authority to a defined higher-authority process. If that process is unavailable or fails to decide within its bounds, the action resolves to DENY.

Escalation governance. The higher-authority process to which ESCALATE transfers decision authority is itself subject to these invariants. It receives a governance evaluation with its own canonical input, delegation artefacts, and policy identity, and produces its own evidence record documenting the transfer and its resolution. ESCALATE does not create a governance gap; it creates a governed delegation of decision authority. Implementations must enforce bounded escalation depth. An escalation chain that exceeds its depth bound resolves to DENY. The escalation, its resolution, and the final outcome are recorded in the evidence chain.

Failure mode. Without explicit decisions, governance is ambiguous. Implicit approval — where the absence of denial is treated as permission — creates a structurally fail-open system. Every undefined case becomes an unintended authorisation.

Dependencies. Requires Invariant 1. Required by Invariants 6, 7.

3.4.2 Authority

Invariant 4 (Delegation)**.** Agents do not possess inherent authority. Authority must be derived exclusively from explicit delegation artefacts that are scoped, time-bound, revocable, and versioned. Without a valid delegation context, the action is denied.

Formal property. No agent possesses inherent authority. An agent's effective authority is exactly the set of valid delegation artefacts presented for evaluation. Each artefact must be:

(i) *Scoped*: specifies the operations, resources, and contexts it authorises.
(ii) *Time-bound*: temporal validity is explicit and bounded.
(iii) *Revocable*: revocation status is resolved before evaluation, cryptographically verifiable, and includes freshness bounds. Evaluation denies if revocation information is absent, freshness bounds are undeclared, status is unverifiable, or the validity window has been exceeded.
(iv) *Versioned*: the delegation artefact carries a version identifier.

If no valid artefact is present in D, then $\mathcal{G}(I, D, P) = \text{DENY}$.

Agent identity. Delegation targets must have stable, verifiable identity. If the identity of an agent cannot be deterministically derived from its composition — model version, system prompts, tools, policy bindings, fine-tuning artefacts, retrieval configuration — delegation validation cannot succeed. Where the runtime cannot provide composition-derived identity, this constitutes a governance gap that must be disclosed, not silently accepted. Disclosure of the gap does not satisfy the delegation requirement; it documents non-satisfaction.

Delegation chains. Sub-delegation must preserve or narrow the authority scope of the delegator. For every artefact $\delta \in D$ and every artefact δ' directly sub-delegated from δ, the sub-delegation must have equal or narrower scope:

$$\forall\, \delta \in D,\ \forall\, \delta' \in \mathit{sub}(\delta) :\ \mathit{scope}(\delta') \subseteq \mathit{scope}(\delta)$$

where $\mathit{sub}(\delta)$ is the set of artefacts directly sub-delegated from δ, and $\mathit{scope}(\delta)$ is the authority scope component s of δ (Definition 2). The full delegation chain from each artefact to its root delegator must be presented and validated during evaluation. Implementations must enforce a bounded chain depth.

Failure mode. Without explicit delegation, authority is ambient — inferred from identity, role labels, or runtime context. Ambient authority is the structural condition that produces confused deputies [17], programs that exercise their own authority on behalf of less-privileged callers. In autonomous AI systems, ambient authority means that any agent with tool access exercises whatever authority the tool's runtime permissions provide, regardless of whether that authority was delegated for the agent's current purpose.

Dependencies. Requires Invariant 1. Required by Invariants 5, 11.

3.4.3 Evaluation and Evidence

Invariant 5 (Deterministic Evaluation)**.** Policy evaluation is a pure function of the canonical action, validated delegation artefacts, and policy identity. The evaluation outcome does not

depend on wall-clock time, external network calls, hidden mutable state, model randomness, non-deterministic execution, or any ambient runtime state that an independent verifier cannot reproduce.

Formal property. $\mathcal{G}$ is pure (Definition 4), and satisfies replay equivalence (Property 1) and canonical replay determinism (Property 2).

All context required for evaluation must be presented as part of the canonical input. The resolution phase (Definition 5) transforms raw runtime context — including temporal validity, revocation status, and sequence-derived facts — into the canonical input bundle. Resolution may perform I/O; evaluation must not. Moving side effects from evaluation to resolution does not exempt them from evidence requirements.

Canonicalisation. Canonicalisation rules and the canonical serialisation format must be explicitly specified, versioned, and identified by an immutable canonicalisation specification identifier included in the evidence record. Changing the canonicalisation specification constitutes a governance change subject to Invariant 10: canonicalisation determines which facts enter the canonical input and how they are serialised, so an untracked change to the canonicalisation specification can silently alter evaluation outcomes or break replay determinism without any change to the policy or decision logic.

Failure mode. Without deterministic evaluation, governance decisions cannot be independently reproduced. A system where identical inputs produce different decisions without a policy change has failed: the decision is a function of hidden state, not of the declared inputs. Such a system cannot be audited, because the auditor cannot reconstruct the conditions under which the decision was made.

Dependencies. Requires Invariants 2, 4. Required by Invariant 6.

Invariant 6 (Evidence Integrity)**.** Every evaluation produces an evidence record. Governance that cannot be proven is indistinguishable from governance that did not occur.

Formal property. For every evaluation $\mathcal{G}(I, D, P) = d$, the system produces an evidence record e satisfying:

(i) *Structured and append-only*: e is a structured record appended to an immutable, verifiably ordered structure.

(ii) *Tamper-evident*: evidence records are cryptographically chained using collision-resistant hash functions. If digital signatures are used, they satisfy unforgeability under chosen-message attack (EUF-CMA) [16].

(iii) *Sufficient for reconstruction*: e contains the canonical action I (including resolved context), delegation artefacts D, policy identity P, canonicalisation specification identifier, the decision d, and the chain linkage proof. This is sufficient to replay the decision deterministically and reproduce the same canonical replay payload for identical inputs.

(iv) *Independently verifiable*: verification does not depend on the originating runtime or vendor infrastructure. The trust root is explicitly defined, independently inspectable, and independent of the system under verification.

(v) *Consequence markers (outcome records)*: for every external consequence, the enforcement boundary emits a corresponding immutable outcome record cryptographically linked to the decision record that authorised execution, enabling independent completeness verification. Outcome records must be emitted by, or verifiably derived from, the enforcement boundary that produces the consequence. This requirement is intentionally strong: in third-party or distributed environments where the execution substrate is not under governance control, satisfying it may require trusted instrumentation not present in current systems. The requirement reflects the position that governance without verifiable consequence binding is assertion, not enforcement.

If an action can produce external consequences without a corresponding evidence record and no mechanism exists to detect the omission, the system cannot establish complete governed coverage of consequence-producing paths and therefore fails anti-bypass (Invariant 2).

Failure mode. Without evidence integrity, governance is assertion. An organisation claims "we evaluated every action" but cannot prove it. A missing or corrupted evidence record constitutes governance failure. Best-effort logging does not satisfy this invariant: operational logs are useful for debugging, but they are neither verifiable independent of the system that executed the action nor sufficient to survive adversarial scrutiny.

Dependencies. Requires Invariants 2, 3, 5. Required by Invariants 10, 11.

3.4.4 System Properties

Invariant 7 (Fail-Closed Enforcement)**.** If governance cannot validate delegation, cannot evaluate policy, detects integrity corruption, or encounters evaluation ambiguity, the action is denied. Governance decisions are authoritative, not advisory. Integrity takes precedence over availability.

Formal property. For any condition ϕ that prevents confident governance evaluation, the system must deny the action:

$$\phi \implies d = \text{DENY}$$

Here d denotes the system-level decision outcome, which may be produced by $\mathcal{G}$ or by the enforcement boundary (contrast Definition 4, where $d = \mathcal{G}(I, D, P)$ specifically). Failure conditions include: inability to reproduce canonical inputs, validate policy identity, verify delegation artefacts, or validate evidence integrity. This is realised through two mechanisms. For conditions expressible within a well-formed canonical input bundle, $\mathcal{G}$ itself returns DENY (Definition 4). For conditions that prevent the bundle from being formed or evaluation from completing — inability to construct (I, D, P), runtime timeout, enforcement boundary unavailability — the enforcement boundary must deny the action before execution. Partial evaluation is not evaluation.

Enforcement is mandatory: execution paths capable of producing external consequences must enforce governance outcomes before execution proceeds. A governance decision that can be ignored by the execution path it governs is not enforcement.

Failure mode. A system that silently degrades governance to maintain availability is not governed; it is optimistic. The alternative — fail-open under governance failure — means the system is ungoverned precisely when it is most likely under attack or experiencing the conditions that make governance most necessary.

Dependencies. Requires Invariant 3.

Invariant 8 (Separation of Planes)**.** Governance must remain structurally distinct from identity systems, policy engines, model guardrails, logging infrastructure, network controls, and agent frameworks.

Formal property. The test is behavioural. Let S be any adjacent system — identity provider, policy engine, model version, logging pipeline, network configuration, or agent framework. If S is upgraded, reconfigured, or replaced, and no explicit governance configuration change is made, then $\mathcal{G}$ must produce the same decisions for the same canonical inputs as before the change to S. If outcomes change without a governance change, the planes are not separated.

Implicit dependency pins, ambient version locks, or undocumented couplings do not constitute explicit governance changes. Note that a change to an adjacent system that changes the composition-derived agent identity (Invariant 4) produces a different canonical input I (specifically, a different α); the governance evaluation function $\mathcal{G}$ correctly produces different decisions

for different inputs. This is not a separation violation but a legitimate governance-input change that should be disclosed under Invariant 10.

Failure mode. When governance collapses into an adjacent system, its semantics are implicitly redefined by changes in that system. Upgrading a model version silently changes what is allowed. Reconfiguring a network control inadvertently bypasses a governance constraint. The system still produces decisions and evidence records, but the meaning of those decisions has changed without any governance change being recorded — a form of drift (Invariant 10) induced by architectural coupling.

Dependencies. None (structural property).

Invariant 9 (Provider Neutrality)**.** Governance semantics must not depend on a specific model provider, agent framework, runtime environment, transport protocol, or infrastructure vendor.

Formal property. Replacing a provider, framework, or infrastructure component is a specific case of adjacent-system change and must satisfy the same behavioural test as Invariant 8. Provider substitution is elevated to a separate invariant because it is a common source of implicit governance coupling in autonomous AI systems. If a vendor substitution silently alters what is allowed, governance cannot honestly be audited as a stable control.

Failure mode. Provider-dependent governance means that switching from one model provider to another, or from one agent framework to another, changes authorisation outcomes without any governance change being made. The evidence chain is intact but meaningless: it proves that actions were governed under conditions that no longer hold.

Dependencies. Extends Invariant 8.

Invariant 10 (Drift Prohibition)**.** Governance must not evolve through undocumented behaviour. All changes to policy semantics, delegation structure, decision logic, and evidence format must be explicit, versioned, and reviewable. Historical evidence records must remain interpretable under their original policy identity.

Formal property. An evidence record e issued under policy version P_k must remain replayable under the policy definitions referenced by P_k to produce an identical decision, regardless of subsequent policy evolution. If replaying a historical record under the policy definitions referenced by its original policy identity produces a different outcome, the evidence chain is broken.

Failure mode. Implicit evolution — governance that changes behaviour without a versioned, reviewable change record — is drift. Drift is governance failure made gradual: the system still produces decisions and evidence records, but the meaning of those decisions silently changes under operational pressure.

Dependencies. Requires Invariant 6.

Invariant 11 (Composition Closure)**.** Governance of individual systems does not guarantee governance of their composition. When governed systems interact, the interaction itself constitutes an execution path subject to governance. Authority that emerges from composition must be governed. This invariant does not introduce a separate evaluation model; it requires that composition-induced execution paths satisfy the same invariants defined for individual actions.

Formal property. Let $Gov(X)$ denote that system X satisfies Invariants 1–10 on every consequence-producing execution path within X's governance boundary. For any composition S of interacting systems $S_1, S_2, \ldots, S_n$:

$$Gov(S)$$

The composition S — including all execution paths induced by component interaction — must satisfy Invariants 1–10. In particular, authority paths — sequences of interactions through which

authority becomes exercisable across components — that no individual delegation authorised must be subject to governance evaluation. Evidence chains across interacting systems must be linkable through a shared trace identifier or coordination artefact so that cross-system decisions can be independently reconstructed. Where composition governance cannot be achieved, the gap must be explicitly disclosed; silent acceptance of ungoverned composition violates this invariant.

Non-closure. *Gov* is not compositional:

$$(\forall\, i :\ Gov(S_i)) \not\Rightarrow Gov(S)$$

Satisfying Invariants 1–10 per-component does not establish $Gov(S)$. Composition can create consequence-producing execution paths that cross no individual component's governance boundary, requiring explicit composition-level governance.

Failure mode. Per-action governance of individual components creates a false sense of security when those components interact. Agent A is authorised to read sensitive data; Agent B is authorised to send external communications. Individually, each delegation is safe. When both coordinate through shared state, the composition creates an authority path — read sensitive data, transmit externally — that no single delegation authorised and no per-agent governance evaluation detects. The system is "governed" at the component level and ungoverned at the composition level.

Dependencies. Directly requires Invariants 2, 4, 6. Composition-induced paths must themselves satisfy Invariants 1–10.

Invariant 12 (Constitutional Supremacy)**.** These invariants override implementation convenience, performance optimisation, vendor capability, and operational expediency. A system that cannot maintain these invariants under load does not have a scaling problem; it has a governance problem.

Formal property. When a trade-off arises between governance integrity and any operational concern — convenience, performance, vendor capability, or expediency — governance integrity prevails. Amendment requires an explicit, versioned revision that increments the version identifier, states the date of effect, and produces a change record identifying each invariant modified and the rationale. Amendments must not alter the meaning of evidence records issued under prior versions.

Failure mode. Without supremacy, governance erodes through accommodation. An invariant is relaxed "just for this deployment." A fail-closed requirement is softened "just during the migration." Each relaxation is individually reasonable; collectively, they dissolve the specification. The system still bears the label "governed" while satisfying none of the invariants that give the label meaning.

Dependencies. Meta-invariant; applies to all others.

Joint necessity (structural argument). We argue that the invariants are jointly necessary: satisfying any subset without the rest does not produce governance. Three examples illustrate the pattern.

Representation without delegation (Invariants 1–3 without Invariant 4; downstream controls held fixed for illustration). Every consequence-producing path passes through governance evaluation (1) with a faithful canonical representation (2) yielding an explicit decision (3). The system may also record evidence, deny on operational failure, and preserve governance versioning and separation. But no delegation artefacts are required. Authority is ambient — inferred from identity, runtime context, or tool permissions. The system evaluates every action faithfully and can record those evaluations, but it cannot distinguish authorised from unauthorised exercise

of a capability. This is the confused deputy [17] with better paperwork — and an impeccable audit trail of confused decisions.

Evidence without fail-closed (Invariants 1–6, 8–12 without Invariant 7). Governance evaluates actions before execution (1) with faithful representations (2), explicit decisions (3), explicit delegation (4), deterministic evaluation (5), and tamper-evident evidence (6). Planes are separated (8, 9), changes versioned (10), composition governed (11), and invariants supreme (12). But when governance cannot form the canonical input, validate delegation, or complete evaluation, the action proceeds. The evidence chain faithfully records every action that was governed and is silent about every action that was not. The system is provably governed when it works and silently ungoverned when it fails.

Per-component governance without composition (Invariants 1–10 without Invariant 11). Each individual component satisfies all non-composition invariants on its own action boundaries. But when two governed components interact through shared state, the composition creates authority paths that no individual evaluation assessed. The system is governed at the component level and ungoverned at the composition level.

These examples illustrate the general pattern: each omitted invariant reopens a specific governance failure mode. Table 1 maps the full invariant set to the governance failure reopened by its absence, and Figure 2 traces the structural dependency relationships. The forcing chain captures the pattern:

> *No consequence without evaluation. No evaluation without faithful representation. No authority without explicit delegation. No proof of governance without verifiable evidence. No failure without denial. No composition without governance.*

3.5 Worked Example

To ground the formal model and invariants in a concrete scenario, we trace a single action through the governance pipeline and then show how composition governance detects a cross-system authority violation.

3.5.1 Single-Action Governance

An autonomous financial assistant is delegated authority to execute supplier payments up to $5,000 on behalf of an operations manager. The assistant invokes a payment API to transfer

Table 1: Joint necessity: the governance failure reopened by removing each invariant.

Inv.	Governance failure in its absence
1	Actions execute without prior evaluation; governance is post-hoc analysis
2	Evaluation governs a fiction; determinism amplifies representation error
3	Undefined cases become implicit authorisations; system is structurally fail-open
4	Authority is ambient; confused deputy at tool-composition scale
5	Decisions are irreproducible; independent audit is impossible
6	Governance is assertion without proof; indistinguishable from its absence
7	System is ungoverned precisely when under attack or experiencing failure
8	Adjacent-system changes silently redefine governance semantics
9	Vendor substitution silently alters what is authorised
10	Governance meaning drifts; historical evidence becomes uninterpretable
11	Component governance creates false assurance; composition is ungoverned
12	Governance erodes through accommodation until invariants are nominal

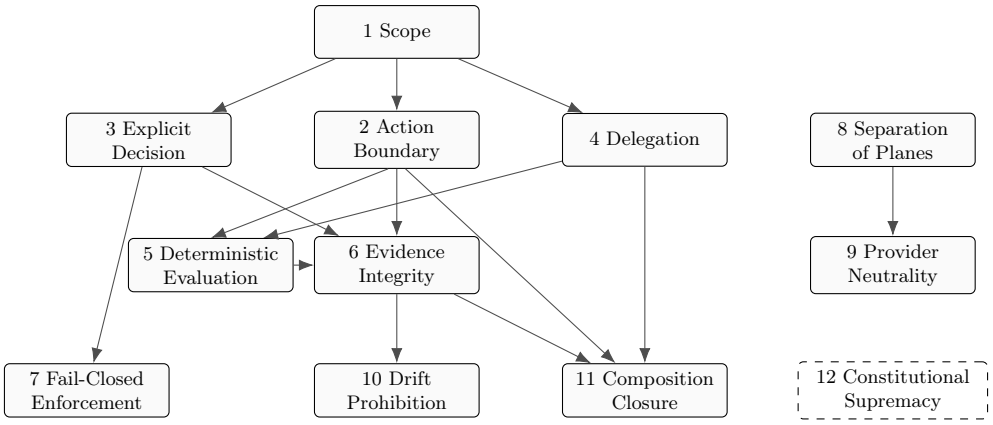

Figure 2: Dependency structure of the twelve constitutional invariants. Arrows point from required to dependent invariant. Invariant 1 (Scope) is foundational; the right column (8, 9) forms an independent chain. Invariant 12 (Constitutional Supremacy) is a meta-invariant constraining all others; it has no dependency edges.

$3,200 to a supplier.

Resolution. The enforcement boundary intercepts the tool call and initiates resolution (Definition 5). Resolution performs I/O: it canonicalises the raw API call into the canonical action I, validates the delegation artefact δ, resolves revocation status against the issuing authority, derives sequence context from prior actions, and acquires a cryptographically authenticated timestamp. The result is the canonical input bundle (I, D, P):

$I = (\alpha,\ \iota,\ \tau,\ \pi,\ \sigma,\ t)$ where:

- α `agent:financial-assistant-v2.1` (composition-derived identity)
- ι `payment.execute`
- τ `account:ops-supplier-payments`
- π `{amount: 3200, currency: USD, recipient: supplier-0442}`
- σ `{prior_action: invoice-read-0441, session_risk: normal}`
- t `2026-04-07T09:14:22Z` (asserted by authenticated time authority)

D One delegation artefact $\delta = (s, b, r, v, c)$:

- s: `payment.execute` on `ops-supplier-payments`, amount $\leq$ `$5000`
- b: `2026-04-01` to `2026-04-30`
- r: `active` (freshness: 12 s)
- v: `deleg-v3`
- c: `operations-manager` $\rightarrow$ `financial-assistant-v2.1`

P `policy:acme-corp-payments-v7`

Evaluation. The governance evaluation function $\mathcal{G}$ receives (I, D, P) and applies the decision logic determined by P. Evaluation is pure: no I/O, no external lookups, no hidden state. The rules check that the attempted action falls within delegation scope: the action type (`payment.execute`) is authorised, the target account matches the delegated resource, and the payment amount ($3,200) is within the delegated limit ($5,000). They also verify temporal validity (within bounds), revocation status (active), and actor-delegation binding (composition-derived identity matches the chain target). All checks pass:

$$d = \mathcal{G}(I, D, P) = \textsc{allow}$$

Enforcement and evidence. The enforcement boundary permits execution and emits a *decision record* containing the full canonical replay payload — I (including σ), D, P, the canonicalisation specification identifier, and $d = \textsc{allow}$ — together with an *outcome record* cryptograph-

ically linked to the decision record and attesting that the payment executed. An independent verifier can replay the decision from that payload and reproduce it byte-for-byte (Property 2).

3.5.2 Composition Governance

Now consider two individually governed agents. Agent A is authorised to read sensitive customer records. Agent B is authorised to send external communications. Each delegation is safe in isolation. But through shared state (a message queue), Agent A reads a customer's financial records and writes them to the queue; Agent B reads from the queue and sends the data to an external address.

Without composition governance, both actions are individually authorised: Agent A's read is within its delegation scope, and Agent B's send is within its delegation scope. Per-action governance of each component produces ALLOW for both.

With composition governance, the resolution phase derives a sequence fact for Agent B's action: σ includes `{derived_from: sensitive-customer-data, prior_agent: A}`. This resolved context is presented as part of Agent B's canonical input I_B. The evaluation function $\mathcal{G}$ now receives a canonical action whose resolved context makes the interaction history explicit. Under a policy that prohibits external transmission of sensitive customer data regardless of which agent originated the read, $\mathcal{G}$ evaluates:

$$d = \mathcal{G}(I_B, D_B, P) = \text{DENY}$$

The denial depends on three conditions acting together: resolution must expose the cross-agent interaction as an explicit resolved fact in σ; the policy must bind meaning to that fact (here, prohibiting external transmission of sensitive data); and evaluation must enforce the prohibition deterministically. No single condition is sufficient — without resolution, the fact is invisible; without the policy, the fact is inert; without deterministic evaluation, the denial is not reproducible. This is the mechanism by which the resolution/evaluation separation (Definition 5) enables composition awareness without collapsing the determinism of evaluation. The resolved fact is explicit, versioned, and recorded in the evidence record; evaluation remains a pure function of its canonical inputs.

This scenario illustrates Invariant 11: governance of Agent A and Agent B individually does not guarantee governance of their composition. The authority path — read sensitive data, then transmit externally — emerges from the interaction and must be governed as an execution path in its own right.

Dependency on resolution trust. The scenario depends on the resolution phase correctly deriving the cross-agent sequence fact. If the adapter for Agent B's environment cannot observe or faithfully represent the data's provenance from Agent A, the resolved context will be incomplete and $\mathcal{G}$ will evaluate a canonical input that omits the governance-relevant interaction history. Where the relevant interaction history cannot be observed or derived, the composition fact remains unresolved and the action must be denied under fail-closed semantics rather than treated as absent. This is precisely the adapter trust problem (Problem 5.3): composition governance is only as reliable as the resolution infrastructure that exposes cross-system facts. The formal model requires these facts; it does not guarantee they can be derived in all deployment topologies.

3.5.3 Escalation

Consider a variant of the payment scenario: the financial assistant attempts a $12,000 payment, exceeding its delegated limit of $5,000. Resolution constructs the same canonical input bundle, but with $\pi =$ `{amount: 12000, ...}`. Evaluation detects that the amount exceeds the delegation scope and the action cannot simply be allowed. However, the policy for this action type specifies that over-limit payments are not categorically prohibited — they require approval from a higher authority (e.g., the CFO). $\mathcal{G}$ evaluates:

$$d = \mathcal{G}(I, D, P) = \text{ESCALATE}$$

The ESCALATE outcome does not execute the payment. It transfers decision authority to a defined higher-authority process — in this case, a human approval workflow bound to the CFO's delegation. The human approver acts as a higher-authority principal within the same governance framework, subject to the same evidence, delegation, and policy requirements as any autonomous actor; human involvement is not an escape from governance but an exercise of authority within it. The escalation itself is governed: the higher-authority process receives its own canonical input (the original action augmented with the escalation context), its own delegation artefacts, and its own policy identity, and produces its own evidence record documenting the transfer, the resolution, and the final outcome. If the CFO approves, the final decision is ALLOW with a two-link evidence chain: the original evaluation record (escalation) and the higher-authority decision record (approval). If the higher-authority process is unavailable or fails to decide within its temporal bounds, the escalation resolves to DENY (Invariant 3).

This scenario illustrates why binary allow/deny is insufficient. A DENY would be incorrect: the delegation does not authorise this amount, but the action is not categorically forbidden. A silent ALLOW would violate the delegation scope. ESCALATE is the structurally correct outcome: the governance system recognises that the decision exceeds its current authority and transfers it to a process that has the requisite authority, without creating a governance gap. The bounded depth requirement prevents infinite escalation chains: if the escalation depth limit (configured per deployment) is reached, the action resolves to DENY. This bound introduces an availability tradeoff: an adversary that can repeatedly trigger escalation-worthy actions may attempt to force denials by exhausting the permitted escalation depth or by saturating the higher-authority path. The model treats this as an availability risk rather than a reason to relax fail-closed semantics: silent authority expansion would be a worse failure than denial. Implementations therefore need explicit mitigations at the escalation layer — bounded authority graphs, cycle detection, escalation-target whitelists, quotas or rate limits on escalation requests, and policy checks that collapse repeated escalation to the same authority path into an immediate DENY rather than unbounded recursion.

4 Landscape Evaluation

We evaluate eleven representative works against the twelve invariants. Table 3 presents the evaluation matrix; this section analyses the structural patterns that emerge.

4.1 Corpus and Scoring Method

The matrix is a criterion-based review of representative works, not an exhaustive meta-analysis of the full literature. We first assembled a working corpus of recent academic papers, implementation artefacts, and institutional documents that make concrete claims about at least one of four

governance dimensions: runtime enforcement, delegation protocols, evidence and integrity, or composition risk. From that corpus, we selected eleven works for side-by-side comparison based on three criteria: architectural specificity, salience in the current discourse, and non-redundancy across the four dimensions. The selection is purposive rather than exhaustive: it is designed to compare distinct architectural approaches, not to estimate frequencies in the underlying literature.

Evaluation principles. A full mark requires that the property is enforced at runtime, not merely described, assumed, or approximated. A partial mark may be awarded when the published artefact demonstrates architectural engagement with the invariant — through mechanisms, design constraints, or formal requirements — at a weaker boundary or on a proper subset of the required properties, even if full runtime enforcement is not demonstrated. In both cases, scores are assigned from the published artefact itself; they do not infer unpublished implementation details, internal deployment practices, or marketing claims beyond what the cited source makes inspectable.

Scoring rule. A full mark (●) requires that all required properties of the invariant, as stated in Section 3.4, be made inspectable in the published artefact at the level of architecture, mechanism, or formal requirement. A partial mark (◉) is assigned when the work addresses a proper subset of those required properties or addresses the invariant at a weaker boundary than required. Where the published artefact does not make the relevant commitment inspectable, the invariant is scored ○ (not satisfied). A missing property is sufficient to prevent a full mark. All scores were assigned by a single rater; inter-rater reliability was not measured. A second independent rater, even on a subset of the corpus, would materially strengthen the evaluation and should be added in a future revision. All scores reflect the published artefacts as of April 2026; subsequent revisions may alter individual scores without affecting the structural patterns reported here.

Per-invariant scoring criteria. To make the scoring reproducible, Table 2 specifies the discriminating properties for each invariant at each score level. A reviewer who disagrees with a particular score can identify the specific property on which the disagreement turns.

4.2 What the field converges on

The strongest convergence is on *pre-execution interception* (Invariant 1). Progent [36], Microsoft AGT [25], Aegis [24], and Kaptein et al. [20] place a policy check before action execution; SAGA [38] applies the same pattern at the inter-agent contact and communication boundary. Chan et al. [6] partially address this through oversight layers that describe action rejection mechanisms; South et al. [37] through credential and scope verification before granting access; Reid et al. [32] through action-level control as an architectural requirement; and Shi et al. [35] by locating policy guards at permission checks in the runtime interaction chain. Together, nine of eleven works engage with the action boundary as a governance surface, but only four implement a general pre-execution action-interception mechanism and one more (SAGA) applies it to agent-to-agent communication. Interception, however, is the easy part. Of those five runtime-enforcement works, none specifies the formal representation properties (completeness, soundness, stability, provenance, binding) that determine whether the intercepted representation faithfully captures the runtime action. Every partial mark on Invariant 2 in the table reflects the same gap: the system evaluates *something*, but cannot characterise the conditions under which that something is faithful.

Partial convergence exists on *explicit decisions* (Invariant 3): five works receive partial marks. Progent, SAGA, and Aegis use binary allow/block-style outcomes; Microsoft AGT exposes a broader decision set including `allow`, `deny`, `warn`, and `require_approval`; and Kaptein et al. output a violation probability with concrete interventions (blocking, human approval) treated as

Table 2: Reproducible scoring rubric. For each invariant, the table states the discriminating criterion between score levels.

Inv.	◍ (partial) requires	● (full) additionally requires
1	Published artefact engages with pre-execution action control (mechanism or architectural requirement)	Every consequence-producing path passes through evaluation before execution
2	Structured action representation evaluated by governance	Representation satisfies all five properties (completeness, soundness, stability, provenance, binding)
3	Inspectable enforcement outcomes that map to allow/deny interventions	Full ALLOW/DENY/ESCALATE trichotomy with ESCALATE failure semantics
4	Delegation addressed (scoping, authority transfer, or attenuation)	Artefacts are scoped, time-bound, revocable, versioned, with cryptographic revocation and chain tracing
5	Deterministic evaluation function stated as design constraint	Replay equivalence and byte-identical canonical replay payloads specified
6	Tamper-evident, cryptographically chained, or cryptographically authenticated evidence or accountability structures	Evidence linked to governance evaluation, independently verifiable, sufficient for decision reconstruction
7	Deny-on-failure for the system's own policy checks	All failure conditions (including inability to form inputs or complete evaluation) result in denial
8	Conceptual distinction between governance and adjacent systems	Behavioural test: adjacent-system change without governance change must not alter decisions
9	Framework-agnostic design or standard policy languages	Full behavioural test across provider/vendor substitution
10	Policy versioning or immutability mechanism	All changes explicit, versioned, reviewable; historical records interpretable under original policy
11	Composition risk identified or partially addressed	Constitutional requirement stated: component compliance $\not\Rightarrow$ system compliance; emergent authority governed
12	Immutable policy or amendment constraints	Invariants override operational expediency; amendment requires versioned revision with change record

a calibration layer — earning a partial mark for producing inspectable enforcement outcomes, though not explicit per-action decisions in the strict sense. None, however, specifies the full ALLOW/DENY/ESCALATE trichotomy or the failure semantics attached to ESCALATE as required by Invariant 3.

Delegation (Invariant 4) is where the field splits. The delegation-focused works — South et al. with what they describe as Agent-ID and Delegation Tokens extending OAuth 2.0/OIDC, and Tomasev et al. [39] with a delegation framework centred on authority transfer, accountability, and privilege attenuation in recursive task delegation — address the mechanism most directly. South et al. specify scope limits, expiry, and revocation endpoints for delegation tokens, but do not describe versioned delegation artefacts, a cryptographic revocation/freshness procedure, or mandatory multi-hop attenuation rules. No work in the corpus fully satisfies this invariant.

Among the enforcement-oriented works, integration remains uneven. SAGA issues task-scoped cryptographic access-control tokens with expiry and request limits, but does not define multi-hop delegation chains or monotonic scope attenuation. Microsoft AGT exposes delegation and scope-chain structures with capability narrowing and expiry, and its policy engine includes an authority-resolution step that can consume delegation context alongside trust and action; the published

Table 3: Evaluation of eleven representative works against the twelve constitutional invariants. Filled circle = invariant satisfied as published; dimmed circle with outline = invariant partially addressed; open circle = invariant not satisfied on the published artefact. Invariant numbers correspond to the definitions in Section 3.4. Works are ordered by publication date.

	Boundary			*Auth.*	*Eval./Evid.*		*System Properties*					
Work	1	2	3	4	5	6	7	8	9	10	11	12
Chan et al. [6]	◐	○	○	○	○	○	○	○	○	○	◐	○
South et al. [37]	◐	○	○	◐	○	○	○	○	◐	○	○	○
Progent [36]	●	◐	◐	○	◐	○	◐	○	○	○	○	○
SAGA [38]	◐	◐	◐	◐	○	○	◐	○	○	○	○	○
Reid et al. [32]	◐	○	○	○	○	○	○	○	○	○	◐	○
Kao [19]	○	○	○	○	○	◐	○	○	◐	○	○	○
Shi et al. [35]	◐	○	○	○	○	○	○	○	○	○	◐	○
Tomasev et al. [39]	○	○	○	◐	○	○	○	○	◐	○	◐	○
Kaptein et al. [20]	●	◐	◐	○	◐	○	○	◐	○	◐	◐	○
Aegis [24]	●	◐	◐	○	○	◐	●	◐	○	◐	○	◐
Microsoft AGT [25]	●	◐	◐	◐	◐	◐	◐	◐	◐	◐	○	○

artefact still does not present a single default end-to-end delegation pipeline satisfying all of the invariant's requirements. Chan et al. discuss identity, certification, and scoped autonomy, but do not specify delegation artefacts. Progent and Aegis do not present delegation artefacts as part of their enforcement path. The result is a field where delegation protocols are advancing faster than source-clean, fully specified governance integration.

Partial convergence also exists on *deterministic evaluation* (Invariant 5): three works — Progent, Kaptein et al., and Microsoft AGT — state or implement deterministic policy evaluation functions, but none specifies replay equivalence or byte-identical canonical replay payloads as defined in Properties 1 and 2.

Evidence (Invariant 6) follows the same pattern: Kao [19] specifies constant-size cryptographic evidence structures for workflow events and configurations with game-based security proofs; Aegis implements an immutable logging kernel tied to its publish-loop evaluation; and Microsoft AGT documents Merkle audit trails with hash-chain integrity, inclusion proofs, and policy-decision logging. Chan et al. discuss accountability infrastructure, identity binding, and certification, but do not specify a tamper-evident, cryptographically chained, or cryptographically authenticated evidence structure under the rubric used here. None of the three marked works receives a full score, however, because the published artefacts do not fully specify the canonical input bundle and independently reconstructible decision record required by Invariant 6.

For comparison, Rajagopalan and Rao [31] present authenticated workflows in which operations carry cryptographic proof or are rejected, but frame the mechanism as a protocol-level authenticated-workflow layer rather than the action-evaluation layer required here.

Fail-closed enforcement (Invariant 7) is addressed by four works. Aegis implements autonomous shutdown on governance violation (the only full mark). Progent and SAGA block disallowed or invalid requests at their enforcement boundary, and Microsoft AGT documents fail-closed behaviour in several governance gateways and authorization paths, but none of these three specifies the stronger requirement that all failure conditions — including inability to form canonical inputs or complete evaluation — resolve to denial.

Microsoft AGT [25] now has the broadest invariant coverage in the corpus, receiving either a full or partial mark on ten of the twelve invariants. It spans the widest range across invariant clusters — including delegation context, evidence, conceptual separation from model-safety controls,

provider-neutral design, and policy versioning — but it still does not satisfy the behavioural separation test, and it still does not present a single canonical governance/evidence contract satisfying the full invariants.

Aegis [24] follows with eight of the twelve. It is the only system with more than one full mark (Invariants 1 and 7): it enforces pre-execution verification, produces explicit allow/deny decisions (though without the ESCALATE pathway), implements fail-closed shutdown, and partially addresses constitutional supremacy through immutable policy with quorum amendment. But it omits delegation entirely (Invariant 4) and, despite explicitly distinguishing runtime governance from training-time alignment and application-layer guardrails, still collapses governance, identity, and logging into an integrated kernel rather than satisfying the behavioural separation test. On the model advanced here, breadth without separation remains integration rather than governance.

The remaining system-property invariants — separation of planes, provider neutrality, drift prohibition, composition closure, and constitutional supremacy (Invariants 8–12) — show no full convergence and only scattered partial convergence in the corpus, and are addressed in the next subsection.

4.3 What no reviewed work fully satisfies

Under the scoring rule defined above, ten invariants receive no full satisfaction from any work in the corpus:

Canonical action representation with formal properties (Invariant 2). Several works intercept actions, but none defines completeness, soundness, stability, provenance, and binding as constitutional requirements of the representation itself. This gap means that even systems performing pre-execution evaluation cannot characterise the conditions under which their representation faithfully captures the runtime action.

Explicit decision with full trichotomy (Invariant 3). Five works produce inspectable enforcement outcomes as detailed above, but none implements the ESCALATE pathway or its required failure semantics: that escalation itself is governed, produces its own evidence record, and resolves to DENY if the higher-authority process is unavailable.

Delegation with full constitutional properties (Invariant 4). Despite the delegation mechanisms detailed above, no work fully satisfies the invariant's requirement that delegation artefacts be scoped, time-bound, revocable, versioned, and consumed as first-class inputs to a governance evaluation function. The gap between the strongest partial treatments (South et al. [37], Microsoft AGT [25]) and a full mark lies in versioned delegation artefacts, cryptographic revocation verification, and mandatory scope narrowing through the delegation chain.

Deterministic evaluation as a pure function (Invariant 5). Three works state deterministic evaluation as a design constraint, but none specifies replay equivalence or byte-identical canonical replay payloads. The distinction between a deterministic function and a pure function of canonical inputs producing reproducible evidence is not stylistic; it determines whether independent replay is possible at all.

Evidence integrity connected to governance evaluation (Invariant 6). Three works satisfy structural sub-properties of the evidence invariant (tamper-evidence or cryptographic chaining). None fully specifies the canonical input bundle and independently reconstructible decision record that would permit an independent verifier to replay a governance decision from the evidence alone.

Separation of planes (Invariant 8). No work passes the behavioural test: a change to any adjacent system must not alter governance semantics unless an explicit governance change is made. Microsoft AGT bundles identity, governance, sandboxing, and observability. Aegis [24] collapses governance, identity, and logging into an integrated kernel. SAGA [38] centralises governance, identity, and discovery in a single Provider entity.

Drift prohibition (Invariant 10). Aegis partially addresses this through immutable policy with quorum amendment; Kaptein et al. [20] require policy changes to be versioned in the audit trail; and Microsoft AGT documents policy schema versioning with validation, migration tooling, and deprecation warnings. Xu et al. [41] define "authorisation drift" as a metric quantifying trust sensitivity in multi-agent systems. But no work specifies the stronger requirement that historical evidence records must remain interpretable under their original policy identity.

Composition closure (Invariant 11). Reid et al. [32] catalogue composition risk through failure mode analysis. Shi et al. [35] analyse cascade risk and cross-agent contamination in the trust-authorisation mismatch setting, and Tomasev et al. [39] and Kaptein et al. partially address composition through recursive delegation and path-based analysis respectively. But no work states the constitutional requirement: component compliance does not imply system compliance, and composition-emergent authority must itself be governed.

Provider neutrality (Invariant 9). Four works receive partial marks through framework-agnostic design or transport-layer abstraction, but none passes the full behavioural test required by Invariant 8, of which provider neutrality is a specialisation.

Constitutional supremacy (Invariant 12). Only Aegis partially addresses this through immutable policy with quorum amendment. No other work specifies a mechanism by which governance invariants override operational expediency or by which amendments are versioned and auditable.

4.4 The fragmentation finding

The evaluation reveals a structural pattern: the field fragments governance into subproblems and solves each independently. The runtime-enforcement works in this corpus concentrate on Invariants 1–3 and 7, with only partial or absent treatment of 4–6 and 8–12. The delegation works concentrate on Invariant 4, with little or no treatment of 1–3 and 5–12. The evidence-focused works concentrate on Invariant 6, with only partial or absent treatment of 1–5 and 7–12. The composition- and interaction-risk analyses concentrate on Invariant 11 without supplying an enforcement mechanism.

This fragmentation is not merely an observation about incomplete implementations. It reflects a structural issue: *within the reviewed corpus, fragment-wise satisfaction of governance invariants does not compose into governance.* To see why, consider the integration boundary concretely. A runtime enforcement toolkit (Invariants 1, 3, 7) evaluates actions before execution, but does not consume delegation artefacts as first-class inputs, so an agent without valid delegation passes governance unchallenged. A delegation protocol (Invariant 4) issues scoped, attenuable tokens, but does not enforce pre-execution evaluation, so a delegated agent can bypass the enforcement boundary entirely. A cryptographic evidence structure (Invariant 6) produces tamper-evident records, but if those records are not linked to a canonical action representation evaluated under an identified policy, they prove that *something* was logged, not that a governance decision was made. Each component may be well engineered; none specifies the integration contract that connects them. It is precisely at that integration boundary — where the enforcement toolkit must consume the delegation artefact, and the evidence structure must record the canonical input bundle under the evaluated policy — that governance failures re-enter. The invariant set as a whole is that integration specification.

This is, at the landscape level, the same structural insight that Invariant 11 captures at the system level: governing the parts does not govern the whole.

5 Open Research Problems

The twelve invariants define what governance of autonomous AI action must not violate. Five open problems stand between the specification and verifiable conformance. Each is framed as a research question and connected to the academic traditions from which solutions are most likely to emerge.

5.1 Problem 1: Conformance Specification

Question. What does it mean for an implementation to *satisfy* an invariant, and how can conformance be verified?

The invariants are stated as properties, not as test procedures. Invariant 5 requires that evaluation be a pure function; Invariant 6 requires tamper-evident evidence chains; Invariant 8 requires a behavioural test against adjacent-system changes. Each requires a different verification methodology. The conformance problem is whether there exists, for each invariant, a relation between specification and implementation that is both operationally checkable and substantively meaningful — analogous to the relationship between a type system and a type checker, or between a security policy and a verified kernel.

The precedent is seL4 [21], which proved that its implementation enforces its capability-based security policy through machine-checked refinement. For the invariants presented here, full formal verification may not always be proportionate to the deployment setting, but the question of what constitutes a *conformance test* for each invariant is open. Replay equivalence (Property 1) is testable: supply identical inputs to two evaluators and compare outputs. Canonical replay determinism (Property 2) is testable: compare canonical replay payload serialisations byte-for-byte. But the behavioural test for separation of planes (Invariant 8) — that no adjacent-system change alters governance semantics without an explicit governance change — is a universally quantified statement over all possible adjacent-system changes, which is not directly testable.

Relevant traditions include process algebra and refinement checking, property-based testing applied to security invariants, and runtime monitoring. The research question is which invariants admit automated conformance verification and which require manual argument or design-level assurance.

5.2 Problem 2: Authority Bootstrap

Question. Every delegation chain terminates at a root. What trust assumptions are necessary and sufficient for the root delegation, and how should they be specified?

Invariant 4 requires that all authority derive from explicit delegation artefacts, that chains be end-to-end traceable, and that sub-delegation preserve or narrow scope. It does not define where the first delegation comes from. This is a deliberate boundary decision: governance evaluates presented delegation artefacts; it does not itself issue root authority. But the invariants depend on the root delegation being trusted under explicitly stated assumptions, which means the trust assumptions at the root propagate through every subsequent delegation.

The problem is structurally analogous to the root-of-trust problem in public-key infrastructure. X.509 [10] addresses certificate validity and revocation, but it does not solve the agent-governance-specific dimensions at issue here: scope constraints that narrow through the chain, revocation state that must be resolved as part of per-action delegation evaluation, and agent identity that may depend on runtime composition rather than solely on certification by an external authority. The OpenID/OAuth delegation literature [37] provides protocol-level mechanisms; the capability tradition [12, 26] provides the theoretical foundation for scope narrowing and confinement. The open problem is specifying the minimal trust assumptions at the root that are sufficient to sustain the invariants throughout the delegation chain, and determining whether those assumptions can be made explicit in a companion specification.

5.3 Problem 3: Adapter and Resolution Trust

Question. The canonical action is a representation of the runtime action and can be lossy whenever the five representation properties are not actually satisfied by the resolution process. Can the five representation properties (completeness, soundness, stability, provenance, binding) be verified at runtime?

This is the primary epistemic limitation of the entire approach. Governance evaluates a *representation* of the action, not the action itself. The representation is produced by a resolution process that transforms raw runtime context into canonical form. If the resolution is incomplete — a tool adds a new parameter that the resolution process does not capture — governance evaluates a structurally valid but substantively incomplete input. The five representation properties (Invariant 2) specify what the canonical representation *must* satisfy, but they are specifications, not proofs.

The gap between specification and verification is real. A system where governance evaluates a canonical action that omits a governance-relevant parameter is governed in form and ungoverned in substance. The adapter that performs the runtime-to-canonical transformation is a member of the trusted computing base: if it fails, everything above it — deterministic evaluation, evidence integrity, anti-bypass — operates on fiction.

Relevant traditions include abstract interpretation (characterising the information loss in a projection), information-flow analysis (tracking which runtime facts reach the canonical representation), and the trusted computing base literature (bounding the components whose failure compromises the system). Runtime attestation — dynamically verifying that the resolution process captures the expected vocabulary — is a promising direction. The research question is whether the representation properties can be made verifiable rather than merely specifiable, and at what cost.

5.4 Problem 4: Composition Governance

Question. How can per-action governance be extended to govern composition without collapsing the determinism of evaluation, and where are the formal limits of that approach?

Invariant 11 states that governance of individual systems does not guarantee governance of their composition, and that composition-emergent authority must be governed. Invariant 5 requires that evaluation be a pure function of canonical inputs. These two invariants create a tension: governing composition requires knowledge of cross-system interactions (state), but evaluation must not depend on hidden mutable state (purity).

The resolution/evaluation separation (Definition 5) is the architectural strategy proposed here: the resolution phase may perform I/O to derive sequence facts, cross-system context, and in-

teraction history, which are then presented as explicit inputs to the pure evaluation function. This separation preserves determinism while admitting compositional awareness. But it raises the question: what are the formal limits of this approach? Can every compositionally dangerous interaction be captured as a resolved fact presented to a per-action evaluator, or are there composition properties that are inherently invisible to per-action governance?

Invariant 11 establishes the requirement: composition-emergent authority must be governed. This problem does not question the requirement; it investigates whether the resolution/evaluation separation is sufficient to satisfy it for all composition properties, or whether additional governance primitives are needed.

The question connects to both trace properties and hyperproperties [8]. A policy such as "no agent may read sensitive data and subsequently transmit externally" is a sequential trace property: it depends on the ordering of actions within an execution, not any single action in isolation. Stronger composition constraints — such as noninterference-style guarantees comparing executions that differ only in secret inputs — are hyperproperties, because they constrain relationships across sets of traces. Whether sequential composition constraints can be enforced through per-action governance with sequence-aware resolution, and where genuinely hyperproperty-level guarantees require a different governance primitive, is an open question. Trace-based verification, hyperproperty analysis, and information-flow control provide the formal apparatus; the research question is whether the resolution/evaluation separation is sufficient to govern the relevant composition constraints through per-action evaluation.

5.5 Problem 5: Adversary Model

Question. Is the adversary model implied by the invariants complete, and how does the resulting threat model differ from traditional access-control threat models?

The invariants clearly presume adversaries: anti-bypass (Invariant 2) assumes an adversary seeking execution paths that evade governance; fail-closed (Invariant 7) assumes an adversary who benefits from governance degradation; evidence integrity (Invariant 6) assumes an adversary who would tamper with or forge evidence records; drift prohibition (Invariant 10) assumes an adversary who benefits from silent semantic change. Section 3.3 gives a preliminary classification of these adversaries, but that classification is not yet shown to be complete or formally sufficient for the invariant set.

The traditional access-control threat model assumes adversaries who attempt to access resources beyond their authorisation. The Dolev-Yao model [13] assumes adversaries who control the communication channel. Capability confinement [26] assumes adversaries who attempt to propagate authority beyond intended scope. The governance threat model for autonomous AI action requires elements of all three, plus adversary classes that are specific to autonomous systems: an adapter that produces unsound representations (Problem 5.3), a composition that creates authority paths no individual delegation authorised (Problem 5.4), and an operator who weakens policy under production pressure until governance becomes permissive enough to be meaningless.

This last adversary class — governance erosion through accommodation — is less explicit in traditional threat models than channel attackers, over-privileged requesters, or confinement breakers, and may be among the most dangerous adversary classes in practice. The research question is whether a formal adversary model can be derived from the invariants and whether the residual attack surface is precisely characterisable. A companion threat model, specifying adversary capabilities and the trust boundaries the invariants assume, would materially improve how the invariants are interpreted and implemented.

Taken together, these five problems define the research boundary between a constitutional specification of governance and a fully worked theory of conformance, trust, and adversarial assurance.

6 Discussion

Adapter trust as epistemic ceiling. The primary limitation of the entire approach is Problem 5.3: governance is only as sound as its representation of reality, and representation is inherently subject to loss or distortion if the representation properties are not satisfied. A system that produces deterministic decisions from a canonical action that omits a governance-relevant parameter is governed in form and ungoverned in substance. This limitation is not unique to the invariants presented here — it applies to any governance system that operates on a representation rather than on the action itself — and must be stated plainly. The five representation properties (Invariant 2) make the requirement explicit; they do not make it automatically satisfied.

Information-flow limitation. The invariants govern actions at the action boundary — the point where intent becomes external consequence through tool invocation, API calls, or state mutations. They do not govern information flows that bypass the action boundary: context-window leakage, inference attacks, semantic encoding in model outputs, or multi-turn accumulation of sensitive information through reasoning alone. These are real governance concerns, but they operate upstream of the action boundary — at the model boundary, where inference shapes what actions are attempted but consequences have not yet been produced. The scope claim is governance of autonomous AI *action* at the point of consequence, not governance of the inference process that generates intent. A complete governance architecture requires both; this paper addresses only the first.

Policy correctness. The invariants guarantee consistent, deterministic, evidence-producing governance. They do not guarantee correct governance. A deterministically evaluated permissive policy produces reproducible but meaningless decisions. A failure mode that may prove as dangerous as governance bypass is governance accommodation: operators who weaken policy under production pressure until the governance layer is present but toothless. This failure mode is outside the scope of the invariants — they define the mechanism, not the policy — but it must inform how implementations are specified, governed, and operated in practice.

Minimality and completeness. The twelve invariants are argued to be jointly necessary on the basis of structural analysis (Table 1) and worked examples (§3.4), not formal proof. We do not prove minimality: some invariants may in principle be derivable from others. Provider neutrality (Invariant 9) is logically a specialisation of separation of planes (Invariant 8), and is stated separately because provider substitution is the most common and practically dangerous source of implicit governance coupling in autonomous AI deployments; the evaluation confirms this: four works receive partial marks on provider neutrality while none satisfies the broader separation test. Constitutional supremacy (Invariant 12) is structurally a meta-invariant rather than a peer, but it is included as a peer constraint because, without it, each of the remaining invariants is individually negotiable under operational pressure, and the derivation (Section 3.4) shows that this negotiation is a distinct governance failure mode not captured by any other invariant. Formal minimality — proving that no invariant is derivable from the conjunction of the others — is future work. We also do not prove completeness: there may be governance properties the invariants do not capture. The information-flow limitation above is one candidate.

Formal model not mechanised. The evaluation function $\mathcal{G}(I, D, P)$ and the resolution/evaluation separation are stated formally but not mechanised in a proof assistant. Replay

equivalence (Property 1) and canonical replay determinism (Property 2) are specified requirements rather than machine-checked theorems. Mechanisation is future work (Problem 5.1).

Concurrent evidence ordering. Invariant 6 requires evidence records to be appended to an immutable, verifiably ordered structure. We clarify the ordering requirement as follows. The minimum ordering requirement is a *verifiable causal ordering*: for any two evaluations e_1, e_2 where the outcome of e_1 is a governance-relevant input to e_2 (e.g., e_1 produces a sequence fact consumed in e_2's resolution), the ordering $e_1 \prec e_2$ must be independently reconstructible from the evidence records alone. Concurrent evaluations with no causal dependency need not be totally ordered; it is sufficient that an independent verifier can determine they are concurrent and that neither depends on the other's outcome. A total order (single append-only sequence, consensus protocol, centralised sequencer) is a sufficient but not necessary implementation choice. In distributed deployments, the requirement may be satisfied by a composition of local append-only sequences plus coordination metadata — Lamport timestamps [22], vector clocks [23], or shared trace identifiers [40] — sufficient for an independent verifier to reconstruct the causal ordering over the evaluations relevant to a decision. The essential property is verifiable completeness: an auditor must be able to determine that no records were omitted or reordered in a way that changes the governance interpretation. Causal ordering is sufficient for this property under the per-action evaluation model presented here, where $\mathcal{G}$ is a pure function of (I, D, P) and no policy rule references concurrent evaluations. Policies with aggregate constraints — rate limits, quotas, or shared-resource locks — may require stronger coordination than causal-order reconstruction alone; such constraints are outside the scope of the current model.

Confidentiality of evidence. Invariant 6 requires evidence records to contain the canonical action, delegation artefacts, and policy identity — sufficient for an independent verifier to replay the decision. In regulated deployments (healthcare, finance, defence), this can mean recording sensitive data — personal identifiers, financial parameters, medical context — in the evidence substrate. The invariant requires that evidence be *sufficient for reconstruction*; it does not require plaintext storage of all inputs. Implementations may satisfy the reconstruction property through cryptographic commitments (hashing inputs rather than recording them in cleartext), redactable signatures, or encrypted evidence with key escrow, provided that reconstruction is achievable when authorised disclosure is granted. The tradeoff between confidentiality and replay verifiability is real and deployment-specific; the invariant specifies the functional requirement (replay must be possible) and leaves the confidentiality mechanism to the implementation.

These limitations define the boundary between specification and implementation and motivate the open problems in Section 5.

Constructive satisfiability. Although this paper does not present a reference implementation as evidence, the invariant set is not merely aspirational. A constructive witness exists at the architectural level. Consider a toy conforming system with six components:

(i) A *boundary adapter* that canonicalises candidate actions into (I, D, P), validates delegation artefacts, resolves revocation status and sequence context, rejects unresolved inputs, and denies actions that fail representation checks.

(ii) A *deterministic evaluator* $\mathcal{G}$ that consumes only canonical artefacts, performs no I/O, and returns exactly one of ALLOW, DENY, or ESCALATE for every input.

(iii) An *enforcement point* that intercepts every consequence-producing path, executes only after ALLOW, blocks on DENY, and routes ESCALATE to a bounded higher-authority workflow that produces its own evidence record.

(iv) An *append-only evidence service* that records the full canonical input bundle, decision, canonicalisation specification identifier, and outcome in a tamper-evident, cryptographically chained sequence, with outcome records linked to the authorising decision record.

(v) A *versioned policy store* that binds every evaluation to an immutable policy identity and

preserves historical policy definitions for replay.

(vi) A *resolution layer* that derives cross-action and cross-agent sequence facts into σ when they are observable, and denies the action when required composition facts cannot be resolved.

Table 4 maps each invariant to the component(s) that satisfy it within this architecture.

Table 4: Satisfiability mapping: each invariant mapped to the component(s) of the toy conforming system that jointly satisfy it. Roman numerals reference the six components defined above.

Inv.	Component(s)	Key property
1	(iii) enforcement point	intercepts every consequence-producing path before execution
2	(i) adapter; (iii) enf. point	canonical representation satisfies five properties; anti-bypass
3	(ii) evaluator; (iii) enf. point	total ALLOW/DENY/ESCALATE; bounded escalation routing
4	(i) adapter; (ii) evaluator	validates scoped, time-bound, revocable, versioned artefacts; denies on $D{=}\emptyset$
5	(ii) evaluator; (iv) evidence	pure function of (I, D, P); byte-identical replay payloads
6	(iv) evidence; (iii) enf. point	tamper-evident chain; outcome records linked to decisions
7	(i), (ii), (iii)	resolution, evaluation, and enforcement failures all deny
8	(ii) evaluator; (v) policy store	$\mathcal{G}$ consumes only (I, D, P); no adjacent-system dependency
9	as 8	provider substitution does not alter inputs or decision logic
10	(v) policy store	all changes versioned; historical records replayable under original P_k
11	(vi) resolution; (ii) evaluator; (i) adapter	composition facts resolved into σ; unresolvable $\Rightarrow$ deny
12	architectural; (v) policy store	invariants override expediency; amendments versioned

This is not a proof of satisfiability, but it is a constructive sketch showing that the invariants are jointly satisfiable at the architectural level without appealing to a particular product or undisclosed mechanism. A reviewer who disputes satisfiability can identify the specific invariant whose mapping they reject.

Implementation experience. The invariants were developed alongside an internal implementation (Disclosure). The existence of an implementation that attempts to satisfy the full invariant set suggests they do not contain obvious internal contradictions that would prevent joint satisfaction, but a single implementation cannot validate a specification. This paper does not present the implementation as an evaluable artefact; the invariants stand or fall on their own terms.

7 Conclusion

Autonomous AI systems now execute actions with irreversible real-world consequences, without per-action human approval. Governing these systems is not a question of adding controls, but of establishing whether control exists at all.

This paper defines twelve constitutional invariants for governing autonomous AI action at the point where intent becomes consequence. The invariants formalise governance as deterministic, pre-execution evaluation over canonical action representations, validated delegation artefacts, and immutable policy identity. They draw from capability-based security and extend it to a setting where authority is dynamic, composition is unbounded, and decisions must be independently verifiable.

The central result is structural: governance does not arise from the accumulation of mechanisms. Systems in the current landscape satisfy fragments of the invariant set, but those fragments do not compose. Under this model, violating any invariant reopens a concrete failure mode in the execution path. Governance is therefore not partial. A system either satisfies the conditions under which actions are authorised before execution, or it does not.

Five open problems — conformance specification, authority bootstrap, adapter trust, composition governance, and adversary modelling — separate constitutional specification from verifiable enforcement. They define the research agenda required to move from architectural necessity to operational assurance.

Governance is not the presence of policy, logging, or control surfaces. It is the ability to determine, before execution and with independently verifiable evidence, whether a specific action was authorised under a specific delegation. The twelve invariants define the conditions under which that determination is possible. Where they are not satisfied, the specific failure mode is identifiable — and the gap between governed appearance and governed reality is measurable.

Disclosure

The author is founder of Ambit Systems. The invariants were developed through that work and are offered here as a public specification; no conformance claims are made about any implementation.

References

[1] C. Adams, P. Cain, D. Pinkas, and R. Zuccherato. Internet X.509 public key infrastructure timestamp protocol (TSP). RFC 3161, 2001.

[2] James P. Anderson. Computer security technology planning study. Technical Report ESD-TR-73-51, Electronic Systems Division, Air Force Systems Command, 1972. Vol. I, October 1972.

[3] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tyre, Ethan Perez, Jamie Kerr, Jared Kaplan, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Karina Nguyen, Nelson Elhage, Nicholas Schiefer, Nisan Stiennon, Nova DasSarma, Oliver Rausch, Robin Larson, Sam McCandlish, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Waldock, Tom Brown, and Tom Henighan. Constitutional AI: Harmlessness from AI feedback, 2022.

[4] Kar Balan, Robert Learney, and Tim Wood. A framework for cryptographic verifiability of end-to-end AI pipelines. https://arxiv.org/abs/2503.22573, 2025. CoRR abs/2503.22573.

[5] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2014. doi: 10.14722/ndss.2014.23212.

[6] Alan Chan, Kevin Wei, Sihao Huang, Nitarshan Rajkumar, Elija Perrier, Seth Lazar, Gillian K. Hadfield, and Markus Anderljung. Infrastructure for AI agents. https://arxiv.org/abs/2501.10114, 2025.

[7] David D. Clark and David R. Wilson. A comparison of commercial and military computer security policies. In *Proceedings of the 1987 IEEE Symposium on Security and Privacy*, pages 184–195, 1987. doi: 10.1109/SP.1987.10001.

[8] Michael R. Clarkson and Fred B. Schneider. Hyperproperties. *Journal of Computer Security*, 18(6): 1157–1210, 2010. doi: 10.3233/JCS-2009-0393.

[9] Common Criteria Recognition Arrangement. Common Criteria for Information Technology Security Evaluation, part 1: Introduction and general model. Standard CCMB-2022-11-001, Common Criteria Recognition Arrangement, 2022. CC:2022 Revision 1, November 2022; published as ISO/IEC 15408-1:2022.

[10] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk. Internet X.509 public key infrastructure certificate and certificate revocation list (CRL) profile. RFC 5280, 2008.

[11] Geoffroy Couprie and Clément Delafargue. Biscuit: A bearer token with offline attenuation and decentralized verification — specification. https://github.com/eclipse-biscuit/biscuit, 2024. Eclipse Biscuit Project, Eclipse Foundation.

[12] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966. doi: 10.1145/365230.365252.

[13] Danny Dolev and Andrew C. Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983. doi: 10.1109/TIT.1983.1056650.

[14] European Parliament and Council. Regulation (EU) 2024/1689 — artificial intelligence act. https://eur-lex.europa.eu/eli/reg/2024/1689/oj, 2024.

[15] Kathleen Fisher, John Launchbury, and Raymond Richards. The HACMS program: Using formal methods to eliminate exploitable bugs. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 375(2104):20150401, 2017. doi: 10.1098/rsta.2015.0401.

[16] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on Computing*, 17(2):281–308, 1988. doi: 10.1137/0217017.

[17] Norm Hardy. The confused deputy. *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988. doi: 10.1145/54289.871709.

[18] M. Jones, J. Bradley, and N. Sakimura. JSON web token (JWT). RFC 7519, 2015.

[19] Leo Kao. Constant-size cryptographic evidence structures for regulated AI workflows. https://arxiv.org/abs/2511.17118, 2025.

[20] Maurits Kaptein, Vassilis-Javed Khan, and Andriy Podstavnychy. Runtime governance for AI agents: Policies on paths. https://arxiv.org/abs/2603.16586, 2026.

[21] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pages 207–220, 2009. doi: 10.1145/1629575.1629596.

[22] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978. doi: 10.1145/359545.359563.

[23] Friedemann Mattern. Virtual time and global states of distributed systems. In *Proceedings of the International Workshop on Parallel and Distributed Algorithms*, pages 215–226, 1988.

[24] Adam Massimo Mazzocchetti. Cryptographic runtime governance for autonomous AI systems: The aegis architecture for verifiable policy enforcement. https://arxiv.org/abs/2603.16938, 2026.

[25] Microsoft. Agent governance toolkit. https://github.com/microsoft/agent-governance-toolkit, 2026. GitHub repository, accessed 2026-04-07, commit 0014d95.

[26] Mark S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006.

[27] National Institute of Standards and Technology. AI agent standards initiative. https://www.nist.gov/caisi/ai-agent-standards-initiative, 2026.

[28] OASIS. eXtensible Access Control Markup Language (XACML) Version 3.0. Oasis standard, OASIS, 2013.

[29] OWASP. OWASP top 10 for agentic applications for 2026. https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/, 2025. Published December 2025; titled for 2026 coverage period.

[30] Sunil Prakash. AIP: Agent identity protocol for verifiable delegation across MCP and A2A. https://arxiv.org/abs/2603.24775, 2026.

[31] Mohan Rajagopalan and Vinay Rao. Authenticated workflows: A systems approach to protecting agentic AI. https://arxiv.org/abs/2602.10465, 2026.

[32] Alistair Reid, Simon O'Callaghan, Liam Carroll, and Tiberio Caetano. Risk analysis techniques for governed LLM-based multi-agent systems. https://arxiv.org/abs/2508.05687, 2025.

[33] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.

[34] Fred B. Schneider. Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000. doi: 10.1145/353323.353382.

[35] Guanquan Shi, Haohua Du, Zhiqiang Wang, Xiaoyu Liang, Weiwenpei Liu, Song Bian, and Zhenyu Guan. SoK: Trust-authorization mismatch in LLM agent interactions. https://arxiv.org/abs/2512.06914, 2025.

[36] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Programmable privilege control for LLM agents. https://arxiv.org/abs/2504.11703, 2025.

[37] Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Dazza Greenwood, Alan Chan, and Alex Pentland. Authenticated delegation and authorized AI agents. https://arxiv.org/abs/2501.09674, 2025.

[38] Georgios Syros, Anshuman Suri, Jacob Ginesin, Cristina Nita-Rotaru, and Alina Oprea. SAGA: A security architecture for governing AI agentic systems. https://arxiv.org/abs/2504.21034, 2025.

[39] Nenad Tomasev, Matija Franklin, and Simon Osindero. Intelligent AI delegation. https://arxiv.org/abs/2602.11865, 2026.

[40] W3C. Trace context. W3C recommendation, World Wide Web Consortium, 2021. 23 November 2021.

[41] Zijie Xu, Minfeng Qi, Shiqing Wu, Lefeng Zhang, Qiwen Wei, Han He, and Ningran Li. The trust paradox in LLM-based multi-agent systems: When collaboration becomes a security vulnerability. https://arxiv.org/abs/2510.18563, 2025.